第11章: 使用技術・ライブラリ完全カタログ — すべての部品の正体

> このアプリは、ゼロから手で書かれた部分だけでできているわけではありません。世界中の技術者が作って公開してくれた「**部品（ライブラリ）**」を大量に組み合わせて作られています。料理にたとえると、本アプリは「自作の料理」ですが、使っている **醤油・出汁・包丁・鍋** はプロのメーカー製です。
>
> この章では、本アプリが使っている **すべての部品** を、`requirements.txt`（バックエンドの部品リスト）と `package.json`（フロントエンドの部品リスト）の **1行ずつ** 取り上げ、「これは何か」「なぜ使うか」「このアプリのどこで使われているか」「バージョンはいくつか」を、初心者にもわかる言葉で解説します。
>
> さらに、この章には **正直な棚卸し** のセクションがあります。本アプリには「リストには書いてあるけれど、実際のコードでは一度も使われていない部品（レガシー・使い残し）」がいくつもあります。後任者が「この部品、消していいの？ 怖くて触れない」と悩まないよう、**実際にコードを調べた結果** をはっきり表にまとめます。これは引き継ぎ資料として最も価値のある情報のひとつです。

この章で学ぶこと

- **ライブラリ（部品）とは何か** — なぜ自分で全部書かないのか
- **パッケージマネージャ** — `pip`（Python）と `npm`（JavaScript）のしくみ
- **バージョン番号の読み方** — `4.2.7` や `^11.14.0` の意味
- **`requirements.txt` 全24行を1行ずつ解説** — Django / DRF / Django-Q / CORS / environ / jsonfield / psycopg2 / Gunicorn / WhiteNoise / openpyxl / pandas ほか
- **`package.json` 全項目を1つずつ解説** — React / TypeScript / Router / axios / MUI+emotion / x-date-pickers+date-fns / chart.js系 / CRA(react-scripts)
- **devDependencies（開発専用の部品）** — Vitest / Testing Library / ESLint / 型定義
- **WSGI / ASGI / Gunicorn の補足** —「Webサーバーとアプリの橋渡し」のしくみ
- **正直な棚卸し** — コードで使われていない疑いのある部品を実調査の結果として一覧化（FullCalendar / react-calendar / dayjs / web-vitals / xlwings / beautifulsoup4 ほか）
- **カレンダーは自前実装** — `generateCalendar` という関数で手作りしている事実
- **不要な依存を安全に整理する手順**

目次

0. 前提知識：ライブラリとパッケージマネージャ
1. バージョン番号の読み方
2. requirements.txt 全24行を1行ずつ解説

3. package.json の構造
4. package.json の dependencies を1つずつ解説
5. package.json の scripts / eslintConfig / browserslist
6. package.json の devDependencies を1つずつ解説
7. settings.py から見るバックエンド部品の使われ方
8. WSGI / ASGI / Gunicorn の補足
9. index.tsx から見るフロント部品の使われ方
10. 正直な棚卸し：使われていない疑いのある部品
11. カレンダーは自前実装という事実
12. 不要な依存を安全に整理する手順
13. トラブルシューティング
14. 演習問題
15. この章のまとめ

---

0. 前提知識：ライブラリとパッケージマネージャ

0.1 ライブラリ（library）とは

> **ライブラリ（library：図書館 → 部品集）とは？** 誰かがあらかじめ作って公開してくれた、**再利用できるプログラムの部品** のこと。あなたのアプリに「取り込んで」使います。図書館で本を借りるように、必要な機能を「借りて」きて、自分のコードから呼び出します。

なぜライブラリを使うのでしょうか。たとえば「日付を画面に表示するカレンダー」を **ゼロから自分で作る** と、うるう年の計算、月ごとの日数、曜日の並び……と、考えることが山ほどあります。これを毎回作るのは大変です。

そこで、世界中の技術者が「カレンダー部品」「グラフ部品」「通信部品」を作って **無料で公開** してくれています。私たちはそれを取り込むだけで、すぐに高機能な画面が作れます。

> **たとえ話:** 家を建てるとき、窓ガラスや蛇口やネジを自分で一から作る人はいません。ホームセンターで既製品を買ってきて組み立てます。ライブラリは「プログラム界のホームセンターの既製品」です。

0.2 パッケージ（package）とパッケージマネージャ

> **パッケージ（package：荷物・梱包）とは？** ライブラリを「配布できる形」に箱詰めしたもの。中身はプログラムのファイル一式と、「この部品はバージョン○○です」「動くにはこの別の部品も必要です」という情報（メタデータ）です。
>
> **パッケージマネージャ（package manager）とは？** パッケージを **自動でダウンロード・インストール・更新・削除** してくれる管理ツール。「この部品が欲しい」と名前を伝えるだけで、必要な関連部品（依存関係）まで芋づる式に集めてくれます。

本アプリでは2種類のパッケージマネージャを使います。

言語	パッケージマネージャ	部品リストのファイル	取得元（倉庫）
Python（バックエンド）	**pip**（ピップ）	`requirements.txt`	PyPI（パイピーアイ）
JavaScript（フロント）	**npm**（エヌピーエム）	`package.json`	npm registry

> **PyPI（パイピーアイ）とは？** Python Package Index の略。Python のパッケージが集まっている世界最大の倉庫サイト（https://pypi.org）。`pip install django` と打つと、ここから Django を取りに行きます。
>
> **npm registry（エヌピーエム レジストリ）とは？** JavaScript のパッケージが集まっている倉庫（https://www.npmjs.com）。`npm install react` でここから React を取りに行きます。

0.3 部品リストファイルの役割

`requirements.txt` や `package.json` は、「**このアプリを動かすのに必要な部品の買い物リスト**」です。

新しいパソコンにこのアプリを入れるとき、次のコマンドを打つだけで、リストに書かれた部品が **すべて自動で揃います**。

> **▼ このコマンドがやること（先に日本語で）：**「`requirements.txt` に書いてある Python 部品を、全部まとめてインストールせよ」という命令です。`pip install -r requirements.txt` でこれらが一括インストールされます。

コピー

```
# -r は「このファイルを読んで全部入れて」の意味
pip install -r requirements.txt
```

> **▼ このコマンドがやること（先に日本語で）：**「`package.json` に書いてある JavaScript 部品を、全部まとめてインストールせよ」という命令です。

コピー

```
# package.json を読んで全部入れる（フロントの frontend/ で実行）
npm install
```

> **なぜ？** 部品リストをファイルに記録しておくと、「私のパソコンでは動くのに、あなたのパソコンでは動かない」という事故を防げます。**全員が同じ部品・同じバージョン** を使えるからです。引き継ぎでも、後任者はこの2ファイルさえあれば環境を再現できます。

---

1. バージョン番号の読み方

部品には必ず **バージョン番号** が付いています。これは「その部品の世代・改訂番号」です。読み方を知らないと、`requirements.txt` も `package.json` も読めません。

1.1 セマンティックバージョニング（意味のあるバージョン番号）

> **セマンティックバージョニング（semantic versioning：意味づけされたバージョン管理。略して SemVer〔セムバー〕）とは？** バージョン番号を `メジャー.マイナー.パッチ` の3つの数字で表す世界共通のルール。たとえば `4.2.7` なら、

コピー

```
4   .   2   .   7
↑       ↑       ↑
メジャー マイナー パッチ
```

部分	名前	上がるとき	たとえ
最初の数字 `4`	**メジャー**	大きな作り変え。**昔のコードが動かなくなる** こともある	車のフルモデルチェンジ
真ん中 `2`	**マイナー**	新機能の追加。**昔のコードはそのまま動く**	オプション装備の追加
最後 `7`	**パッチ**	バグ修正だけ。機能は変わらない	リコール修理

> **なぜ重要？** メジャーが上がる更新（例： `4.x` → `5.x` ）は、**今まで動いていた自分のコードが急に壊れる** 危険があります。だから、うかつに上げてはいけません。逆にパッチ更新（ `4.2.7` → `4.2.8` ）は安全なことが多いです。

1.2 `requirements.txt` の `==`（完全固定）

`requirements.txt` では、次のように書かれています。

コピー

```
Django==4.2.7
```

`==` は「**ぴったりこのバージョン**」という意味です。`4.2.7` 以外は入れません。これを **バージョン固定**（pin：ピン留め）と呼びます。

> **なぜ固定するの？**「動作確認した、まさにそのバージョン」に釘で留めておけば、後で勝手に新しい版が入って壊れる事故を防げます。本アプリの `requirements.txt` は **全部 `==`** で固定されています。これは「再現性を最優先」した堅実な書き方です。

1.3 `package.json` の `^`（キャレット）

`package.json` では、次のように書かれています。

コピー

```
"react": "^19.1.0"
```

先頭の `^`（キャレット〔caret〕、山形記号）は「**この版以上で、メジャーは上げない範囲なら新しくてOK**」という意味です。

コピー

```
^19.1.0  →  19.1.0 以上 20.0.0 未満（19.x.x なら何でもOK）
```

> **用語**: `^19.1.0` は「19系の最新パッチ・最新マイナーまで自動で許可。ただし 20 には上げない」。マイナー（新機能）とパッチ（修正）は受け入れるが、メジャー（破壊的変更）は拒む、という安全寄りの指定です。
>
> ⚠️ `^` は「範囲指定」なので、`npm install` するタイミング次第で実際に入る版が少しずつ変わります。これを完全に固定するために、npm は別途 `package-lock.json`（パッケージ ロックファイル）という「実際に入れた版を1つ残らず記録した台帳」を自動生成します。**この `package-lock.json` も必ず Git に入れて引き継ぐ** ことが、環境を完全再現するコツです。

---

2. requirements.txt 全24行を1行ずつ解説

ここからが本番です。本アプリのバックエンド部品リスト `requirements.txt`（全24行）を、**1行も飛ばさず** 解説します。

> **▼ このファイルがやること（先に日本語で）**：Python 側（Django バックエンド）を動かすために必要な部品を、24個ぜんぶ「名前==バージョン」の形で並べた買い物リストです。`pip install -r requirements.txt` でこれらが一括インストールされます。

実物のファイル全体を、まず丸ごと載せます。

コピー

```
Django==4.2.7
django-bootstrap-datepicker-plus==5.0.5
django-bootstrap4==23.3
django-crispy-forms==2.1
django-environ==0.11.2
django-filter==23.5
django-widget-tweaks==1.5.0
django-widgets-improved==1.5.0
djangorestframework==3.14.0
django-q==1.3.9
gunicorn==21.2.0
openpyxl==3.1.2
psycopg2-binary==2.9.9
python-dateutil==2.8.2
python-dotenv==1.0.0
python-json-logger==2.0.7
xlwings==0.30.13
whitenoise==6.6.0
beautifulsoup4==4.12.2
pandas==2.1.2
python-Levenshtein==0.26.1
setuptools==76.1.0
django-cors-headers==4.7.0
jsonfield==3.2.0
```

それでは1行ずつ見ていきます。各行を「**何か(よみ)/なぜ使うか/このアプリのどこで使うか/バージョン/状態**」の表で整理します。「状態」は、実際にコードで使われているか（◎=中核 / ○=使用 / △=設定にだけ存在 / ×=未使用の疑い）を表します。

2.1 中核フレームワーク

コピー

```
Django==4.2.7
```

項目	内容
何か	**Django（ジャンゴ）** — Python製の代表的なWebアプリ・フレームワーク（土台一式）。"D" は発音せず「ジャンゴ」
なぜ使うか	URLの振り分け、データベース操作（ORM）、管理画面、セキュリティなど、Webアプリに必要な機能が「全部入り」で手に入るから
どこで使う	**アプリ全体の土台。**`kosu/` アプリ、`models.py`、`urls.py`、`views/`、管理画面すべて
バージョン	`4.2.7`（`4.2` は LTS〔長期サポート版〕。安定運用向け）
状態	◎ 中核

> **用語: フレームワーク（framework：枠組み）とは？** ライブラリよりさらに大きい「アプリの骨格そのもの」。ライブラリが「部品」なら、フレームワークは「家の設計済みの枠組み」。あなたはその枠の決められた場所にコードを置いていきます。
>
> **用語: LTS（エルティーエス：Long-Term Support、長期サポート）とは？**「長期間、バグ修正やセキュリティ修正を提供します」と約束されたバージョン。`Django 4.2` はLTSなので、長く安心して使えます。

コピー

```
djangorestframework==3.14.0
```

項目	内容
何か	**Django REST Framework（ジャンゴ レスト フレームワーク。略してDRF〔ディーアールエフ〕）** — Djangoで「API」を作るための拡張部品
なぜ使うか	本アプリはフロント（React）とバック（Django）が分離している。両者が **JSON** でやり取りするための「API」を、DRFが簡単に作れるから
どこで使う	`settings.py` の `REST_FRAMEWORK` 設定、`kosu/views/` の各APIビュー、ページネーション（`PAGE_SIZE: 20`）
バージョン	`3.14.0`
状態	◎ 中核

用語: API（エーピーアイ：Application Programming Interface）とは？ プログラム同士の「窓口・受付」。Reactが「工数一覧ちょうだい」と窓口に頼むと、Djangoが「はいどうぞ」とJSONを返す。その窓口の作りをDRFが担当します。

用語: JSON（ジェイソン：JavaScript Object Notation）とは？ データを `{"name": "山田", "age": 30}` のように「名前と値の組」で表す、軽くて読みやすいテキスト形式。プログラム同士のデータ交換の世界標準です。

コピー

```
django-q==1.3.9
```

項目	内容
何か	**Django-Q（ジャンゴキュー）** — Djangoで「時間のかかる処理を裏で動かす」ための部品（タスクキュー）
なぜ使うか	バックアップやリストアなど、**数十秒〜数分かかる重い処理** を、画面を固まらせず裏側で実行するため
どこで使う	`settings.py` の `Q_CLUSTER` 設定、`kosu/tasks.py`（バックアップ/リストアの非同期タスク）、`AsyncTask` モデル
バージョン	`1.3.9`
状態	◎ 中核

用語: タスクキュー（task queue：仕事の待ち行列）とは？「あとでやる仕事」を行列に並べておき、専用の作業員（ワーカー）が順番に片付けるしくみ。レストランで注文票（キュー）を厨房に回し、コックが順に作るのと同じ。Django-Qの作業員は `python manage.py qcluster` で起動します。

なぜ裏でやるの？ もしバックアップ処理を画面の中で直接やると、終わるまでユーザーの画面が固まってしまいます。「裏で動かして、終わったら教える」方式なら、ユーザーは待たずに他の操作を続けられます。

コピー

```
django-cors-headers==4.7.0
```

項目	内容
何か	**django-cors-headers（コルス ヘッダーズ）** — CORS（クロスオリジン通信）を許可するための部品
なぜ使うか	開発中、フロントは `localhost:3000`、バックは `localhost:8000` という **別の住所** で動く。ブラウザは標準で「別住所への通信」を遮断するため、それを許可する設定が要る
どこで使う	`settings.py` の `MIDDLEWARE` 先頭（`corsheaders.middleware.CorsMiddleware`）、`CORS_ORIGIN_ALLOW_ALL` などの設定
バージョン	`4.7.0`
状態	◎ 中核

> **用語: CORS（コルス：Cross-Origin Resource Sharing、オリジン間リソース共有）とは？**「オリジン（住所＝プロトコル＋ドメイン＋ポート）が違うサイト同士の通信を、どこまで許すか」のルール。ブラウザのセキュリティ機能で、初期状態では別オリジンへの通信を止めます。この部品が「うちのフロントからのアクセスはOK」と通行許可証を出します。

コピー

```
django-environ==0.11.2
```

項目	内容
何か	**django-environ（ジャンゴ エンヴァイロン）** — `.env` ファイルから設定値を読み込む部品
なぜ使うか	パスワードや秘密鍵を **コードに直接書かず**、`.env` という別ファイルに隔離して安全に読み込むため
どこで使う	`settings.py` 冒頭 `import environ` → `env = environ.Env()` → `env.read_env(...)` → `SECRET_KEY=env('SECRET_KEY')` など
バージョン	`0.11.2`
状態	◎ 中核

> **用語: 環境変数（environment variable）／.env ファイルとは？** パスワードやサーバー名など「環境ごとに変わる秘密の設定」を、コードの外（`.env` ファイルや OS の環境変数）に置く考え方。Gitに上げないことで、秘密が漏れるのを防ぎます（詳しくは設定・デプロイの章）。

コピー

```
jsonfield==3.2.0
```

項目	内容
何か	**jsonfield（ジェイソンフィールド）** — DjangoモデルでJSON形式のデータを丸ごと1つの列に保存できるフィールド型
なぜ使うか	構造が複雑なデータ（リストや辞書）を、わざわざ別テーブルに分けず1列にJSONとして保存できて手軽だから
どこで使う	`kosu/models.py` でJSON型の項目を持つモデル（例：履歴やタスクの付随データ）
バージョン	`3.2.0`
状態	○ 使用

コピー

```
psycopg2-binary==2.9.9
```

項目	内容
何か	**psycopg2（サイコピーグ・ツー）** — PythonからPostgreSQLデータベースに接続するためのドライバ。`-binary` は「コンパイル済みで入れやすい版」
なぜ使うか	本アプリのデータベースは PostgreSQL。DjangoがそのDBと会話するには、この「通訳」が必須だから
どこで使う	`settings.py` の `DATABASES` → `'ENGINE': 'django.db.backends.postgresql'`。Djangoが裏でこのドライバを呼ぶ
バージョン	`2.9.9`
状態	◎ 中核

> **用語: データベースドライバ（database driver）とは？** プログラムとデータベースの間に立つ「専用の通訳」。DjangoのORM（Python語）を、PostgreSQLが理解できる言葉に翻訳して橋渡しします。
>
> **用語: PostgreSQL（ポストグレスキューエル、通称ポスグレ）とは？** 高機能で信頼性の高い、オープンソースのリレーショナルデータベース（表形式でデータを保存・検索する倉庫システム）。本アプリのデータ保管庫です。

2.2 本番運用・配信まわり

コピー

```
gunicorn==21.2.0
```

項目	内容
何か	**Gunicorn（ジーユニコーン：Green Unicorn）** — Python製アプリを本番で動かすための「WSGI HTTPサーバー」
なぜ使うか	Django付属の `runserver` は **開発専用**（遅く・非力）。本番では大量のアクセスを安定してさばけるGunicornを使うため
どこで使う	本番デプロイ時の起動（例：`gunicorn hozen_another.wsgi`）。`settings.py` の `WSGI_APPLICATION = 'hozen_another.wsgi.application'` が入口
バージョン	`21.2.0`
状態	◎ 中核（本番）。WSGIの詳細は§8

コピー

```
whitenoise==6.6.0
```

項目	内容
何か	**WhiteNoise（ホワイトノイズ）** — Python（Django）アプリ自身が、CSSや画像などの「静的ファイル」を効率よく配信できるようにする部品
なぜ使うか	本番で静的ファイルを配るのに、通常は別のWebサーバー（Nginx等）が必要。WhiteNoiseを使えば **Django単体で完結** でき、デプロイが簡単になる
どこで使う	`settings.py` の `MIDDLEWARE` に `whitenoise.middleware.WhiteNoiseMiddleware`、`STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'`
バージョン	`6.6.0`
状態	◎ 中核

> **用語: 静的ファイル（static files）とは？** 中身が変化しない配信物。CSS・JavaScript・画像・フォントなど。これに対し、毎回内容が変わるHTML（工数一覧など）は「動的」です。本アプリは `staticfiles/` フォルダにこれらを集めています（ `STATIC_ROOT` ）。
>
> **なぜ「圧縮」マニフェスト？** `CompressedManifestStaticFilesStorage` は、ファイルを `.gz` で圧縮し、ファイル名にハッシュ（ `base.64976e0f7339.css` のような英数字）を付けます。圧縮で通信が速くなり、ハッシュ名で「古いキャッシュが残る事故」を防ぎます。Gitの差分にたくさん出てくる `staticfiles/...` はこの仕組みの産物です。

2.3 データ処理（Excel・表計算）

コピー

```
openpyxl==3.1.2
```

項目	内容
何か	**openpyxl（オープンパイエックスエル）** — PythonでExcelファイル（.xlsx）を読み書きする部品
なぜ使うか	工数データなどをExcel形式で出力（ダウンロード）するため
どこで使う	`kosu/views/team_views.py`（`import openpyxl`）、`kosu/tasks.py`（`import openpyxl`）。**実際にコードで使われている**
バージョン	`3.1.2`
状態	○ 使用

コピー

```
pandas==2.1.2
```

項目	内容
何か	**pandas（パンダス）** — Pythonでデータ（表）を集計・加工する強力な部品。データ分析の定番
なぜ使うか	大量の工数データを表として読み込み、集計・整形するため
どこで使う	`kosu/tasks.py`（`import pandas as pd`）。**実際にコードで使われている**
バージョン	`2.1.2`
状態	○ 使用

> **用語: DataFrame（データフレーム）とは？** pandasの中心的なデータ型で、「Excelのシートのような行と列の表」をPythonの中で扱える形にしたもの。並べ替え・集計・フィルタが数行で書けます。

コピー

```
python-dateutil==2.8.2
```

項目	内容
何か	**python-dateutil（パイソン デートユーティル）** — 日付・時刻の高度な計算（曖昧な日付文字列の解釈、月単位の加算など）を行う部品
なぜ使うか	標準の `datetime` では面倒な日付計算を簡単にするため。ただし pandas が内部で依存しているため、自動で入っている可能性が高い
どこで使う	**本アプリのコードからは直接importされていない**（調査結果）。pandas等の「依存の依存」として入っている可能性大
バージョン	`2.8.2`
状態	△ 直接の使用は確認できず（間接依存の可能性）

> ⚠️ この行は「正直な棚卸し」の対象です。詳しくは§10で扱います。

コピー

```
python-dotenv==1.0.0
```

項目	内容
何か	**python-dotenv（パイソン ドットエンブ）** — `.env` ファイルを読み込む部品（django-environ と役割が重なる）
なぜ使うか	`.env` から環境変数を読み込むため。ただし本アプリの `settings.py` は **django-environ** を使っており、こちらは重複の可能性
どこで使う	`settings.py` では `import environ`（django-environ）を使用。python-dotenv の直接importは確認できず
バージョン	`1.0.0`
状態	△ 重複の疑い（§10参照）

コピー

```
python-json-logger==2.0.7
```

項目	内容
何か	**python-json-logger（パイソン ジェイソン ロガー）** — ログをJSON形式で出力するための部品
なぜ使うか	ログを機械処理しやすいJSON形式にするため
どこで使う	`settings.py` の `LOGGING` 設定は `formatter: simple`（`'format': '{message}'`）であり、**JSONフォーマッタは使っていない**。直接importも確認できず
バージョン	`2.0.7`
状態	✕ 未使用の疑い（§10参照）

コピー

```
xlwings==0.30.13
```

項目	内容
何か	**xlwings（エックスエルウイングス）** — PythonからExcelアプリ本体を **自動操作** する部品（Excelを起動して操作する）
なぜ使うか	Excelの自動操作が必要なら使う。ただし本アプリのExcel処理は openpyxl で行っており、xlwings は不要の可能性
どこで使う	コード内で `import xlwings` は **見つからない**（調査結果）。さらに xlwings は Windows/Mac の Excel アプリ常駐前提で、Linuxサーバー環境では動かない
バージョン	`0.30.13`
状態	✕ 未使用の疑い（§10参照）

コピー

```
beautifulsoup4==4.12.2
```

項目	内容
何か	**Beautiful Soup 4（ビューティフル スープ フォー。略して bs4）** — HTMLやXMLを解析（スクレイピング）する部品
なぜ使うか	Webページから情報を取り出す（スクレイピング）用途。本アプリにその機能は見当たらない
どこで使う	`import bs4` / `from bs4` は **見つからない**（調査結果）
バージョン	`4.12.2`
状態	✕ 未使用の疑い（§10参照）

コピー

```
python-Levenshtein==0.26.1
```

項目	内容
何か	**python-Levenshtein（レーベンシュタイン）** — 2つの文字列の「似ている度合い（編集距離）」を高速計算する部品
なぜ使うか	あいまい検索や類似文字列の判定に使う。本アプリにその処理は見当たらない
どこで使う	`import Levenshtein` は **見つからない**（調査結果）
バージョン	`0.26.1`
状態	✕ 未使用の疑い（§10参照）

> **用語: 編集距離（Levenshtein distance：レーベンシュタイン距離）とは？** 文字列Aを文字列Bにするのに必要な「1文字の挿入・削除・置換」の最小回数。たとえば「kitten」→「sitting」は3回。検索ワードの打ち間違いを救う「もしかして検索」などに使います。

コピー

```
setuptools==76.1.0
```

項目	内容
何か	**setuptools（セットアップツールズ）** — Pythonパッケージのインストール・ビルドを支える基盤部品
なぜ使うか	多くのパッケージが内部でこれに依存する「縁の下の力持ち」。バージョンを固定して互換性事故を防ぐため明示している
どこで使う	アプリコードからは直接使わない。pip/各種パッケージの土台
バージョン	`76.1.0`
状態	△ 基盤（直接importはしない）

2.4 旧テンプレート時代の名残（Bootstrap/フォーム系）

ここからの6つは、**Reactに移行する前、Djangoのテンプレート（HTML）で画面を作っていた時代の部品** と考えられます。本アプリは今や画面をReactで作っているため、これらの多くは「過去の地層」です。ただし一部は `settings.py` に登録が残っています。

コピー

```
django-bootstrap-datepicker-plus==5.0.5
```

項目	内容
何か	**django-bootstrap-datepicker-plus** — Djangoのフォームに「日付選択カレンダー」ウィジェットを追加する部品
なぜ使うか	（旧）Djangoテンプレートの入力フォームに日付ピッカーを出すため
どこで使う	`settings.py` の `INSTALLED_APPS` に `'bootstrap_datepicker_plus'` として登録は **残っている**。だがReact画面では使われていない
バージョン	`5.0.5`
状態	△ 設定にだけ存在（実フロントは未使用）

コピー

```
django-bootstrap4==23.3
```

項目	内容
何か	**django-bootstrap4** — DjangoテンプレートでCSSフレームワーク「Bootstrap 4」を使いやすくする部品
なぜ使うか	（旧）Djangoテンプレートの見た目をBootstrapで整えるため
どこで使う	`settings.py` の `INSTALLED_APPS` に `'bootstrap4'`、`TEMPLATES` の `builtins` に `'bootstrap4.templatetags.bootstrap4'`、`BOOTSTRAP4 = {'include_jquery': True}` として **残存**。React画面の見た目はこれと無関係（独自CSS）
バージョン	`23.3`
状態	△ 設定にだけ存在（実フロントは未使用）

コピー

```
django-crispy-forms==2.1
```

項目	内容
何か	**django-crispy-forms（クリスピー フォームズ）** — Djangoのフォームを綺麗なHTMLに整形する部品
なぜ使うか	（旧）テンプレートのフォーム表示を美しくするため
どこで使う	`INSTALLED_APPS` にも登録されておらず、コードでの使用も確認できない
バージョン	`2.1`
状態	× 未使用の疑い（§10参照）

コピー

```
django-filter==23.5
```

項目	内容
何か	**django-filter** — 一覧APIに「絞り込み（フィルタ）」機能を簡単に付ける部品
なぜ使うか	URLのクエリで一覧を絞り込むため
どこで使う	`INSTALLED_APPS` に未登録、`DEFAULT_FILTER_BACKENDS` 設定もなし。コードでの使用も確認できない
バージョン	`23.5`
状態	× 未使用の疑い（§10参照）

コピー

```
django-widget-tweaks==1.5.0
```

項目	内容
何か	**django-widget-tweaks** — Djangoテンプレート内でフォーム部品の属性（class等）を後から微調整できる部品
なぜ使うか	（旧）テンプレートのフォーム見た目を細かく調整するため
どこで使う	`INSTALLED_APPS` に未登録、テンプレートでの `{% load widget_tweaks %}` も見当たらない
バージョン	`1.5.0`
状態	× 未使用の疑い（§10参照）

コピー

```
django-widgets-improved==1.5.0
```

項目	内容
何か	**django-widgets-improved** — Djangoのフォームウィジェットを改善する部品（widget-tweaks に近い）
なぜ使うか	（旧）テンプレートのフォーム改善のため
どこで使う	`INSTALLED_APPS` に未登録、使用も確認できない
バージョン	`1.5.0`
状態	× 未使用の疑い（§10参照）

> **▼ ここまでで分かること（先に日本語で）：** `requirements.txt` の24個のうち、**確実に中核・使用中** なのは Django / DRF / Django-Q / cors-headers / environ / jsonfield / psycopg2 / gunicorn / whitenoise / openpyxl / pandas の約11個。残りの多くは「Reactへ移行する前の名残」や「依存の依存」で、**直接は使われていない疑い** があります。詳しい根拠と整理手順は §10・§12 で扱います。

---

3. package.json の構造

次はフロントエンドです。`package.json` は、`requirements.txt` よりも構造が複雑なので、まず **全体の地図** を押さえます。

> **▼ このファイルがやること（先に日本語で）**：フロント（React）の「名前・バージョン・必要部品・実行コマンド・対応ブラウザ」を、ひとつのJSONにまとめた設定ファイルです。npm はこれを読んでアプリを管理します。

`package.json` は大きく次のブロックに分かれます。

コピー

```
{
  "name": "frontend",           // ← このプロジェクトの名前
  "version": "0.1.0",           // ← このプロジェクト自身のバージョン
  "private": true,              // ← npmの公開倉庫に誤って公開しない安全フラグ
  "dependencies": { ... },      // ← 本番でも必要な部品（アプリの動作に必須）
  "scripts": { ... },           // ← npm run ○○ で動かす定型コマンド集
  "eslintConfig": { ... },      // ← コードの書き方チェックの設定
  "browserslist": { ... },      // ← どのブラウザに対応するかの指定
  "devDependencies": { ... }    // ← 開発・テスト時だけ必要な部品
}
```

各行を見ていきます。

コピー

```
"name": "frontend",
```

このプロジェクトの **名前**。`frontend` という素っ気ない名前ですが、内部識別用なので問題ありません。

コピー

```
"version": "0.1.0",
```

このプロジェクト **自身** のバージョン。`0.1.0` は「まだ正式版1.0前の開発段階」を示す慣習的な初期値です。アプリ機能とは無関係。

コピー

```
"private": true,
```

> **なぜ true？** これを `true` にしておくと、間違って `npm publish` （npmの公開倉庫にアップロード）を実行しても**公開されません**。社内アプリのコードを世界に晒さないための安全装置です。
>
> **dependencies と devDependencies の違い（重要）：- dependencies（ディペンデンシーズ：依存）** … アプリが動くために本番でも必要な部品（React本体、通信ライブラリなど）。- **devDependencies（デブ ディペンデンシーズ：開発時依存）** … 開発・テスト中だけ必要で、完成品（ビルド結果）には含まれない部品（テストツール、コード検査ツールなど）。
>
> たとえるなら、dependencies は「料理の材料（皿に乗る）」、devDependencies は「調理器具（皿には乗らない）」です。

---

4. package.json の dependencies を1つずつ解説

`dependencies` ブロック（本番でも必要な部品）を全部解説します。実物は次の通りです。

コピー

```
"dependencies": {
  "@emotion/react": "^11.14.0",
  "@emotion/styled": "^11.14.1",
  "@fullcalendar/daygrid": "^6.1.19",
  "@fullcalendar/interaction": "^6.1.19",
  "@fullcalendar/react": "^6.1.19",
  "@fullcalendar/timegrid": "^6.1.19",
  "@mui/material": "^7.2.0",
  "@mui/x-date-pickers": "^8.11.2",
  "@testing-library/dom": "^10.4.0",
  "@types/react": "^19.1.7",
  "@types/react-dom": "^19.1.6",
  "axios": "^1.11.0",
  "chart.js": "^4.5.0",
  "chartjs-plugin-datalabels": "^2.2.0",
  "date-fns": "^4.1.0",
  "dayjs": "^1.11.13",
  "react": "^19.1.0",
  "react-calendar": "^6.0.0",
  "react-chartjs-2": "^5.3.0",
  "react-dom": "^19.1.0",
  "react-router-dom": "^7.11.0",
  "react-scripts": "5.0.1",
  "typescript": "^4.9.5",
  "web-vitals": "^2.1.4"
}
```

> **用語: スコープ付きパッケージ（scoped package）とは？** `@emotion/react` のように `@組織名/部品名` の形をした名前。`@` で始まるのは「特定の組織・チームが管理する部品群」という印です。たとえば `@mui/...` はMUIチーム、`@fullcalendar/...` はFullCalendarチームの部品です。

4.1 React 本体まわり（◎ 中核）

コピー

```
"react": "^19.1.0",
```

項目	内容
何か	**React（リアクト）** — Meta（Facebook）製の、画面を「部品（コンポーネント）」で組み立てるUIライブラリ
なぜ使うか	データが変わったら画面の必要な部分だけを自動で書き換えてくれる。本アプリの全画面がReact製
どこで使う	全 `.tsx` ファイル。`index.tsx` 冒頭 `import React from 'react';`
バージョン	`^19.1.0`（最新世代のReact 19）
状態	◎ 中核

コピー

```
"react-dom": "^19.1.0",
```

項目	内容
何か	**ReactDOM（リアクト ドム）** — Reactで作った部品を **実際のブラウザ画面（DOM）に描く** 橋渡し部品
なぜ使うか	React本体は「画面の設計図」を作るだけ。それをブラウザの画面に反映するのがReactDOMの役目
どこで使う	`index.tsx` の `import ReactDOM from 'react-dom/client';` と `ReactDOM.createRoot(...)`
バージョン	`^19.1.0`（Reactと同世代に揃える）
状態	◎ 中核

> ⚠️ **react と react-dom は必ず同じメジャーバージョンに揃える** こと。片方だけ上げると噛み合わず壊れます。本アプリは両方 `^19.1.0` で揃っており正しい状態です。

コピー

```
"react-router-dom": "^7.11.0",
```

項目	内容
何か	**React Router DOM（リアクト ルーター ドム）** — URLに応じて表示する画面を切り替える「ルーティング」部品
なぜ使うか	`/login` ならログイン画面、`/kosu-list` なら工数一覧、と **URLと画面を結びつける** ため
どこで使う	`index.tsx` の `import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom";` と、その下の `<Routes><Route ... /></Routes>` 全体
バージョン	`^7.11.0`
状態	◎ 中核

> **用語: ルーティング（routing）とは？** 「URL（住所）」と「表示する画面」を対応づけること。郵便の住所ごとに配達先を決めるのと同じ。本アプリでは約50個のURLが `index.tsx` の `<Route>` で定義されています（§9で実物を見ます）。

コピー

```
"react-scripts": "5.0.1",
```

項目	内容
何か	**react-scripts** — Create React App（CRA）の中身。開発サーバー起動・ビルド・テスト設定を「全部入り」で提供する道具箱
なぜ使うか	Webpack・Babel・ESLintなどの **面倒な設定を自分で書かずに済む** から。`npm start` `npm run build` の正体がこれ
どこで使う	`scripts` の `start` `build` `eject` が `react-scripts ○○` を呼ぶ
バージョン	`5.0.1`（`^` なしで完全固定。下手に上げると壊れやすいため固定が無難）
状態	◎ 中核（ビルド基盤）

> **用語: Create React App（クリエイト リアクト アップ。略してCRA）とは？** Reactアプリの「初期セット」を一発で用意してくれる公式の足場ツール。本アプリはこれで作られています。ただしテスト実行は後述のVitestに置き換えられています（`scripts` の `test` が `vitest` なのが証拠）。
>
> **用語: ビルド（build）とは？** 人間が書いたソースコード（TypeScript / JSX）を、ブラウザがそのまま読める形（普通のJS・CSS・HTML）に **変換・圧縮してまとめる** こと。結果は `frontend/build/` に出ます。

コピー

```
"typescript": "^4.9.5",
```

項目	内容
何か	**TypeScript（タイプスクリプト）** — JavaScriptに「型（データの種類のラベル）」を足した言語。`.tsx` `.ts` を書ける
なぜ使うか	「数値のはずの場所に文字を入れた」等のミスを **動かす前に** 発見できるから
どこで使う	フロントの全ソース（`.tsx` `.ts`）。コンパイラ本体がこれ
バージョン	`^4.9.5`（TypeScript 4系。React 19のおすすめは5系だが4系で運用している）
状態	◎ 中核

> **用語: 型（type）とは？**「この変数は数値」「これは文字列」というデータの種類のラベル。型を付けておくと、間違った使い方をエディタが赤線で警告してくれます（詳細はTypeScriptの章）。

4.2 見た目の部品（MUI + emotion）

コピー

```
"@mui/material": "^7.2.0",
```

項目	内容
何か	**MUI（エムユーアイ：Material UI）** — Google発「マテリアルデザイン」に沿った、完成度の高いReact用UI部品集（ボタン・入力欄・ダイアログなど）
なぜ使うか	綺麗で操作性のよい部品を一から作らず使えるから
どこで使う	多数の画面（時刻ピッカーの土台、各種UI部品）
バージョン	`^7.2.0`（MUI v7）
状態	○ 使用

コピー

```
"@emotion/react": "^11.14.0",
"@emotion/styled": "^11.14.1",
```

項目	内容
何か	**emotion（エモーション）** — JavaScriptの中にCSSを書ける「CSS-in-JS」ライブラリ。`react` 版と `styled` 版の2点セット
なぜ使うか	**MUIが内部でemotionを必須としている** ため。MUIを使うなら必ず一緒に入れる相棒
どこで使う	MUI部品の見た目を支える土台。直接書かなくてもMUIが裏で使う
バージョン	`^11.14.0` / `^11.14.1`
状態	○ 使用（MUIの必須相棒）

> **なぜ2つ要るの？** `@emotion/react` が基本機能、`@emotion/styled` が「`styled(Button)` のようにスタイル付き部品を作る」機能。MUIはこの両方を前提に作られているので、セットで必要です。

4.3 日付・時刻ピッカー

コピー

```
"@mui/x-date-pickers": "^8.11.2",
```

項目	内容
何か	**MUI X Date Pickers** — MUIの「日付・時刻を選ぶ高機能部品」集
なぜ使うか	休憩時間や工数入力で「時刻をパッと選ぶ」UIを出すため
どこで使う	`KosuNew.tsx` / `KosuEdit.tsx` / `BreakTime.tsx` / `TodayBreakTime.tsx` の `MobileTimePicker` `LocalizationProvider` `AdapterDateFns`（実コードで確認）
バージョン	`^8.11.2`（MUI X v8）
状態	○ 使用

コピー

```
"date-fns": "^4.1.0",
```

項目	内容
何か	**date-fns（デートエフエヌエス）** — 日付の整形・計算をする軽量ライブラリ（関数の集まり）
なぜ使うか	MUIの日付ピッカーが「日付の計算エンジン」を必要とし、その役を date-fns が担うため
どこで使う	上記4ファイルの `import { AdapterDateFns } from "@mui/x-date-pickers/AdapterDateFns";`。`AdapterDateFns` が裏で date-fns を呼ぶ（実コードで確認）
バージョン	`^4.1.0`
状態	○ 使用

> **用語: アダプタ（adapter：変換器）とは？** MUIのピッカーは「日付ライブラリを差し替え可能」に作られていて、`AdapterDateFns` は「date-fnsをピッカーに接続する変換プラグ」です。だから `date-fns` 本体と `AdapterDateFns` がセットで必要になります。

コピー

```
"dayjs": "^1.11.13",
```

項目	内容
何か	**Day.js（デイジェイエス）** — date-fns と同じく日付を扱う軽量ライブラリ（別系統）
なぜ使うか	日付ライブラリのもう1つの選択肢。だが本アプリは date-fns を採用しており、dayjs は出番がない
どこで使う	`import ... from 'dayjs'` は **src内に見つからない**（調査結果）
バージョン	`^1.11.13`
状態	✕ 未使用の疑い（§10参照）

> ⚠️ `date-fns` と `dayjs` は **同じ目的の競合ライブラリ** です。両方は要りません。本アプリで実際に使われているのは date-fns 側です。

4.4 通信（axios）

コピー

```
"axios": "^1.11.0",
```

項目	内容
何か	**axios（アクシオス）** — ブラウザからサーバーへHTTP通信（データのお願い）を送る部品
なぜ使うか	React（フロント）がDjango（バック）のAPIに「データちょうだい」「保存して」と頼むため。本アプリ通信の主役
どこで使う	`frontend/src/api/axios.ts`（共通設定）と、ほぼ全ページが import（調査で全ページ多数の使用を確認）
バージョン	`^1.11.0`
状態	◎ 中核

> **用語: HTTP通信／リクエスト・レスポンスとは？** ブラウザとサーバーの会話の作法。ブラウザが「○○ください」と頼むのが **リクエスト**、サーバーが「はいどうぞ」と返すのが **レスポンス**。axiosはこの会話を代行する「電話係」です。
>
> **なぜ専用ファイル `api/axios.ts` ？** 通信先のベースURLや「401（未ログイン）が返ったらログイン画面へ飛ばす」といった共通設定を1か所にまとめるためです（API通信の章で詳説）。

4.5 グラフ（chart.js 系）

コピー

```
"chart.js": "^4.5.0",
```

項目	内容
何か	**Chart.js（チャートジェイエス）** — 棒グラフ・円グラフなどを描く定番の描画エンジン
なぜ使うか	工数を集計して **棒グラフで見える化** するため
どこで使う	`KosuTotal.tsx` / `Components/KosuBarChart.tsx`（react-chartjs-2 経由で利用）
バージョン	`^4.5.0`
状態	○ 使用

コピー

```
"react-chartjs-2": "^5.3.0",
```

項目	内容
何か	**react-chartjs-2** — Chart.js を「Reactの部品」として使えるようにする橋渡し
なぜ使うか	Chart.js は単体だと素のJS向け。Reactで `<Bar />` のように書くにはこのラッパーが要る
どこで使う	`KosuTotal.tsx` / `KosuBarChart.tsx` の `import { Bar } from "react-chartjs-2";`（実コードで確認）
バージョン	`^5.3.0`
状態	○ 使用

コピー

```
"chartjs-plugin-datalabels": "^2.2.0",
```

項目	内容
何か	**chartjs-plugin-datalabels** — Chart.jsの各棒・各点に「数値ラベル」を直接表示する拡張プラグイン
なぜ使うか	棒グラフの上に実数値を出して読みやすくするため
どこで使う	`KosuTotal.tsx` の `import ChartDataLabels from "chartjs-plugin-datalabels";`（実コードで確認）
バージョン	`^2.2.0`
状態	○ 使用

> **用語: ラッパー（wrapper：包むもの）とは？** ある部品を「別の流儀で使えるように包んだ部品」。react-chartjs-2 は Chart.js を React 流に包んだラッパーです。

4.6 カレンダー（FullCalendar / react-calendar）

コピー

```
"@fullcalendar/daygrid": "^6.1.19",
"@fullcalendar/interaction": "^6.1.19",
"@fullcalendar/react": "^6.1.19",
"@fullcalendar/timegrid": "^6.1.19",
```

項目	内容
何か	**FullCalendar（フルカレンダー）** — 月表示・週表示・予定のドラッグなど高機能なカレンダー部品（4つの小部品セット）
なぜ使うか	本来はリッチなカレンダーUIを出すための部品
どこで使う	`@fullcalendar/...` の import は **src内に1件も見つからない**（調査結果）。本アプリのカレンダーは自前実装（§11）
バージョン	すべて `^6.1.19`
状態	✕ 未使用の疑い（§10・§11参照）

コピー

```
"react-calendar": "^6.0.0",
```

項目	内容
何か	**react-calendar** — シンプルな月カレンダー部品（FullCalendarより軽量）
なぜ使うか	カレンダー表示の別候補
どこで使う	`import ... from 'react-calendar'` は **src内に見つからない**（調査結果）
バージョン	`^6.0.0`
状態	✕ 未使用の疑い（§10・§11参照）

> ⚠️ カレンダー部品が **3系統（FullCalendar 4点 + react-calendar）も入っている** のに、実際の画面は自前の `generateCalendar` 関数で作っています。これは「いろいろ試した跡」が残った状態です。§11で実物を確認します。

4.7 型定義・テスト関連がdependenciesに混在

コピー

```
"@types/react": "^19.1.7",
"@types/react-dom": "^19.1.6",
```

項目	内容
何か	**@types/react / @types/react-dom** — React と ReactDOM の **TypeScript用「型情報」だけのパッケージ**
なぜ使うか	React本体はJSで書かれ型情報を持たない。TypeScriptが型チェックするために、この別売り型定義が必要
どこで使う	コンパイル時にTypeScriptが参照（コードに直接importはしない）
バージョン	`^19.1.7` / `^19.1.6`（React 19に合わせる）
状態	○ 使用（厳密には開発時用だが、ここでは dependencies に置かれている）

コピー

```
"@testing-library/dom": "^10.4.0",
```

項目	内容
何か	**@testing-library/dom** — テストでDOM（画面要素）を操作・検証する土台ライブラリ
なぜ使うか	テスト用部品（ `@testing-library/react` ）が内部で必要とする土台
どこで使う	テストコード（直接importは少なく、react版の土台として働く）
バージョン	`^10.4.0`
状態	○ 使用（本来は開発時依存の性質）

> ⚠️ `@types/...` や `@testing-library/dom` は本来 devDependencies に置くのが理想ですが、本アプリでは dependencies に入っています。**動作には支障ありません**（npm はどちらも入れる）。整理する場合の注意は§12で触れます。

4.8 web-vitals

コピー

```
"web-vitals": "^2.1.4",
```

項目	内容
何か	**web-vitals** — Googleが定める「表示速度などの体験指標（LCP/FID/CLS等）」を計測する部品
なぜ使うか	CRAの初期テンプレートに付属。性能計測用
どこで使う	`frontend/src/reportWebVitals.ts` 内でのみ参照。だが **その reportWebVitals 自体が `index.tsx` から呼ばれていない**（調査結果）ため、計測は実際には作動していない
バージョン	`^2.1.4`
状態	× 未使用の疑い（§10参照）

---

5. package.json の scripts / eslintConfig / browserslist

dependencies の次の3ブロックを解説します。

5.1 scripts（定型コマンド集）

コピー

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "vitest",
  "test:ui": "vitest --ui",
  "test:run": "vitest run",
  "eject": "react-scripts eject"
},
```

> **▼ このブロックがやること（先に日本語で）：** `npm run ○○` と打ったときに実際に走る「ショートカット定義」です。長いコマンドに短い名前を付けています。

1行ずつ。

コピー

```
"start": "react-scripts start",
```

`npm start` で実行 → 開発サーバーが立ち上がり `http://localhost:3000` で画面が見られる。コードを保存すると自動で画面が更新（ホットリロード）。

コピー

```
"build": "react-scripts build",
```

`npm run build` で実行 → 本番用に最適化（圧縮・まとめ）したファイルを `frontend/build/` に出力。これを実サーバーに置く。

コピー

```
"test": "vitest",
```

`npm test` で実行 → テストツール **Vitest** を「監視モード」で起動（ファイルを保存するたびに自動でテスト）。CRA標準のJestではなく **Vitestに差し替え** ている点が重要。

コピー

```
"test:ui": "vitest --ui",
```

`npm run test:ui` で実行 → Vitestを **ブラウザ上の見やすい画面** で動かす。

コピー

```
"test:run": "vitest run",
```

`npm run test:run` で実行 → テストを **1回だけ** 実行して結果を出し終了（CI＝自動検査向き）。

コピー

```
"eject": "react-scripts eject",
```

`npm run eject` で実行 → CRAが隠している内部設定を **すべて外に取り出す** 一方通行の操作。

> ⚠️ **`eject` は絶対に安易に実行しないこと。** 一度実行すると元に戻せません。CRAの隠し設定が全部表に出てきて、以後ぜんぶ自分で面倒を見る羽目になります。引き継ぎ後、よほどの理由がない限り触らないでください。
>
> **用語: ホットリロード（hot reload）とは？** コードを保存した瞬間、ブラウザを手動で更新しなくても画面が自動で書き換わる開発支援機能。試行錯誤が速くなります。

5.2 eslintConfig（コードの書き方チェック）

コピー

```
"eslintConfig": {
  "extends": [
    "react-app",
    "react-app/jest"
  ]
},
```

> **▼ このブロックがやること（先に日本語で）**：ESLint（コードの書き方を自動チェックする道具）の設定です。CRA公式の推奨ルールセットを「そのまま使う」と宣言しています。

- `"react-app"` … CRA公式の標準ルール集（Reactで一般的な書き方の作法）。
- `"react-app/jest"` … テストコード向けの追加ルール集。

> **用語: ESLint（イーエスリント）とは？** コードを実行する前に「未使用の変数がある」「書き方が危ない」等を指摘してくれる **静的解析ツール（リンター）**。校正者が原稿の誤字を赤入れするのに似ています。`extends` は「既存のルール集を継承して使う」という意味です。

5.3 browserslist（対応ブラウザの範囲）

コピー

```
"browserslist": {
  "production": [
    ">0.2%",
    "not dead",
    "not op_mini all"
  ],
  "development": [
    "last 1 chrome version",
    "last 1 firefox version",
    "last 1 safari version"
  ]
},
```

> **▼ このブロックがやること（先に日本語で）**：「どのブラウザまで動くようにビルドするか」の指定です。ビルドツールがこれを見て、古いブラウザ向けの変換をどこまでするか決めます。

本番（production）の指定の意味：

- `">0.2%"` … 世界の利用シェアが **0.2%超** のブラウザを対象にする。
- `"not dead"` … もう更新が止まった「死んだ」ブラウザは除外。
- `"not op_mini all"` … Opera Mini（機能が限られる軽量ブラウザ）は除外。

開発（development）の指定の意味：

- `"last 1 chrome version"` … Chromeの **最新1バージョン** だけを対象（開発中は最新だけ見れば十分なので速くなる）。Firefox・Safariも同様。

> **用語: browserslist（ブラウザリスト）とは？**「対応ブラウザの範囲」を一元管理する共通設定。Babel（変換ツール）やCSSの自動補完ツールがこの設定を読んで、必要な互換コードだけを生成します。広く対応するほど変換が増えてファイルが大きくなるため、本番と開発で範囲を分けています。

---

6. package.json の devDependencies を1つずつ解説

開発・テスト時だけ必要な部品です。完成品（build結果）には入りません。

コピー

```
"devDependencies": {
  "@testing-library/jest-dom": "^6.9.1",
  "@testing-library/react": "^16.3.1",
  "@types/jest": "^30.0.0",
  "@vitejs/plugin-react": "^5.1.2",
  "@vitest/coverage-v8": "^4.1.2",
  "@vitest/ui": "^4.0.16",
  "eslint-config-react-app": "^7.0.1",
  "jsdom": "^27.3.0",
  "vitest": "^4.0.16",
  "webpack-bundle-analyzer": "^4.10.2"
}
```

1つずつ。

コピー

```
"@testing-library/jest-dom": "^6.9.1",
```

項目	内容
何か	**jest-dom** — テストで「画面に○○が表示されているか」を自然な言葉で検証する追加機能（`toBeInTheDocument()` 等）
なぜ使うか	「この要素は画面にあるか」を読みやすく書けるから
どこで使う	テストのセットアップ（`frontend/src/tests/` 配下）
状態	○ 使用（テスト用）

コピー

```
"@testing-library/react": "^16.3.1",
```

項目	内容
何か	**React Testing Library（RTL）** — Reactコンポーネントを「ユーザー目線」でテストする中心ライブラリ
なぜ使うか	「ボタンを押したらこう表示される」を、実際の使い方に近い形でテストできる
どこで使う	各テストファイルの `render(...)` `screen` など
状態	◎ 中核（テスト）

コピー

```
"@types/jest": "^30.0.0",
```

項目	内容
何か	**@types/jest** — Jest風のテスト関数（`describe` `it` `expect` 等）のTypeScript型定義
なぜ使うか	テストコードで型補完・型チェックを効かせるため
どこで使う	テストコードのコンパイル時
状態	○ 使用（テスト用型）

コピー

```
"@vitejs/plugin-react": "^5.1.2",
```

項目	内容
何か	**@vitejs/plugin-react** — Vite/Vitest が React（JSX）を理解するためのプラグイン
なぜ使うか	VitestがReactのテストを動かすのに必須
どこで使う	Vitestの設定ファイル（`vitest.config` 等）
状態	○ 使用（テスト基盤）

コピー

```
"@vitest/coverage-v8": "^4.1.2",
```

項目	内容
何か	**Vitest カバレッジ（coverage：網羅率）** —「テストがコードの何%を通ったか」を計測する部品（V8エンジン利用）
なぜ使うか	テストの抜け漏れを数値で把握するため
どこで使う	カバレッジ計測実行時
状態	○ 使用（テスト用）

> **用語: カバレッジ（coverage：網羅率）とは？** 全コードのうちテストで実際に実行された割合。「80%」なら2割は一度もテストで動いていない、という目安。高いほど安心の度合いが上がります（テストの章で詳説）。

コピー

```
"@vitest/ui": "^4.0.16",
```

項目	内容
何か	**Vitest UI** — テスト結果をブラウザの画面で見られる部品
なぜ使うか	`npm run test:ui` で結果を視覚的に確認するため
どこで使う	`test:ui` スクリプト
状態	○ 使用（テスト用）

コピー

```
"eslint-config-react-app": "^7.0.1",
```

項目	内容
何か	**eslint-config-react-app** — §5.2 の `eslintConfig` で継承していた `react-app` ルールの **本体**
なぜ使うか	コード検査ルールの実体だから
どこで使う	ESLint実行時
状態	○ 使用（検査ルール）

コピー

```
"jsdom": "^27.3.0",
```

項目	内容
何か	**jsdom** — Node.js（ブラウザのない実行環境）の中に「仮想のブラウザ画面（DOM）」を作る部品
なぜ使うか	テストはブラウザなしで走るが、画面操作の検証には「画面のフリ」が要る。それを用意する
どこで使う	Vitestのテスト環境設定（`environment: 'jsdom'`）
状態	◎ 中核（テスト環境）

用語: Node.js（ノードジェイエス）とは？ ブラウザの外（パソコンやサーバー上）でJavaScriptを動かす実行環境。テストやビルドはこの上で走ります。Node.jsには画面（DOM）がないため、jsdomが「画面のダミー」を提供します。

コピー

```
"vitest": "^4.0.16",
```

項目	内容
何か	**Vitest（ヴィテスト）** — 高速なテスト実行ツール。Jestの後継的な存在
なぜ使うか	本アプリのテストの中心エンジン。`npm test` の正体
どこで使う	全テスト実行（`scripts` の `test` `test:ui` `test:run`）
状態	◎ 中核（テスト）

コピー

```
"webpack-bundle-analyzer": "^4.10.2"
```

項目	内容
何か	**webpack-bundle-analyzer** — ビルド結果のファイルサイズを「どの部品が何KB占めるか」見える化する分析ツール
なぜ使うか	アプリが重いとき、どの部品が容量を食っているか調べるため
どこで使う	必要時に手動で実行（常用ではない）
状態	△ 補助ツール（必要時のみ）

7. settings.py から見るバックエンド部品の使われ方

`requirements.txt` の部品が **実際にどこで効いているか** を、`settings.py` の該当箇所と突き合わせます。リストとコードを結びつけて理解するのが目的です。

7.1 INSTALLED_APPS（Djangoに登録するアプリ／部品）

> **▼ この設定がやること（先に日本語で）**：「このDjangoプロジェクトで使う機能（アプリ）」をDjangoに登録する一覧です。ここに名前を書いて初めて、その部品が有効になります。

コピー

```
INSTALLED_APPS = [
    # Django標準：管理画面
    'django.contrib.admin',
    # Django標準：ユーザー認証
    'django.contrib.auth',
    # Django標準：モデルの種類管理（内部基盤）
    'django.contrib.contenttypes',
    # Django標準：セッション（ログイン状態の保持）
    'django.contrib.sessions',
    # Django標準：一時メッセージ表示
    'django.contrib.messages',
    # Django標準：静的ファイル管理
    'django.contrib.staticfiles',
    # ← django-bootstrap4（旧テンプレート時代の名残）
    'bootstrap4',
    # ← django-bootstrap-datepicker-plus（同上・名残）
    'bootstrap_datepicker_plus',
    # ← 本アプリ本体（自作）
    'kosu',
    # ← Django-Q（非同期タスク：中核）
    'django_q',
    # ← DRF（API：中核）
    'rest_framework',
    # ← django-cors-headers（CORS許可：中核）
    'corsheaders',
]
```

各行を見ると、`requirements.txt` のどの部品が「登録されているか」が分かります。

- `'kosu'` … 自作アプリ本体（ライブラリではない）。
- `'django_q'` `'rest_framework'` `'corsheaders'` … 中核部品。確かに登録されている。
- `'bootstrap4'` `'bootstrap_datepicker_plus'` … 登録は **残っている** が、React画面では出番がない（§10）。

> **なぜ「名残」が残っているの？** Reactに移行する前のテンプレート時代の登録を消し忘れているためと考えられます。**登録が残っているだけなら害は小さい** ですが、混乱の元なので整理候補です（§12）。

7.2 MIDDLEWARE（リクエストを順に通すフィルタ列）

> **▼ この設定がやること（先に日本語で）**：受け取ったリクエストを「上から順に通す関所（ミドルウェア）」の並びです。CORS許可・セキュリティ・セッション・静的ファイル配信などが、ここで順番に処理されます。

コピー

```
MIDDLEWARE = [
    # ← cors-headers：別オリジン通信を許可（一番先頭が重要）
    'corsheaders.middleware.CorsMiddleware',
    # Django標準：セキュリティ
    'django.middleware.security.SecurityMiddleware',
    # Django標準：セッション
    'django.contrib.sessions.middleware.SessionMiddleware',
    # Django標準：共通処理
    'django.middleware.common.CommonMiddleware',
    # Django標準：CSRF対策
    'django.middleware.csrf.CsrfViewMiddleware',
    # Django標準：認証
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    # Django標準：メッセージ
    'django.contrib.messages.middleware.MessageMiddleware',
    # Django標準：クリックジャッキング対策
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    # ← WhiteNoise：静的ファイル配信
    'whitenoise.middleware.WhiteNoiseMiddleware',
    # ← 自作ミドルウェア
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
]
```

ここで `requirements.txt` の cors-headers と whitenoise が実際に効いているのが確認できます。

> **用語: ミドルウェア（middleware：中間処理）とは？** リクエストとアプリ本体の「間」に挟まる処理の層。空港でいう「保安検査 → パスポート確認 → 搭乗」のような一連の関所。`corsheaders` を一番上に置くのは「最初にCORSのヘッダを付ける」必要があるためです。

7.3 REST_FRAMEWORK（DRFの設定）

コピー

```
REST_FRAMEWORK = {
    # 一覧をページ分割する方式
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    # 1ページ20件
    'PAGE_SIZE': 20,
    'DEFAULT_PARSER_CLASSES': (
        # 受信はJSONだけ受け付ける
        'rest_framework.parsers.JSONParser',
    ),
    'DEFAULT_RENDERER_CLASSES': (
        # 送信はJSONで返す
        'rest_framework.renderers.JSONRenderer',
    ),
}
```

これは `djangorestframework`（DRF）の設定です。「一覧は20件ずつページ分け」「やり取りはJSONのみ」と決めています。リストの DRF がここで具体的に働いています。

7.4 Q_CLUSTER（Django-Qの設定）

コピー

```
Q_CLUSTER = {
    # タスクの管理にデフォルトDBを使う
    'orm': 'default',
    # 作業員（同時処理）は4人
    'workers': 4,
    # 2000タスクごとに作業員をリフレッシュ
    'recycle': 2000,
    # 失敗時の再試行までの秒数
    'retry': 2000,
    # 1タスクの上限時間（1800秒＝30分）
    'timeout': 1800,
}
```

これは `django-q` の設定。「裏方の作業員を4人用意し、最大30分かかる重い処理まで面倒を見る」という意味です。リストの Django-Q が具体的に効いています。

7.5 LOGGING（python-json-logger は使われていない証拠）

コピー

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'simple': {
            # ← ただのメッセージ文字列。JSON形式ではない
            'format': '{message}',
            'style': '{',
        },
    },
    # …（handlers / loggers が続く）…
}
```

ここに注目。フォーマッタは `'simple'`（`'format': '{message}'`）であり、**JSON形式ではありません**。つまり `requirements.txt` の `python-json-logger`（JSONでログを出す部品）は **このアプリでは使われていない** ことが、設定からも裏付けられます（§10で再掲）。

---

8. WSGI / ASGI / Gunicorn の補足

`requirements.txt` の Gunicorn、`settings.py` の `WSGI_APPLICATION` を理解するには、「WSGI」という言葉を押さえる必要があります。

8.1 そもそもWebサーバーとアプリは別もの

ブラウザからのアクセスは、まず **Webサーバー** が受け取ります。しかしWebサーバーは「Pythonアプリを直接動かす方法」を知りません。逆にDjango（Pythonアプリ）も「生のネットワーク通信のさばき方」は専門外です。

そこで、両者をつなぐ **共通の約束ごと** が必要になります。それが **WSGI** です。

> **用語: WSGI（ウィズギー：Web Server Gateway Interface）とは？**「PythonのWebアプリ」と「Webサーバー」の **会話のお作法を統一した規格**。コンセントの形を世界共通にするように、「この形でデータを渡せば、どのサーバーとどのPythonアプリも噛み合う」と決めたもの。

8.2 Gunicornの役割

> **▼ ここでやること（先に日本語で）**：Gunicornが「Webサーバーからのリクエスト」を受け取り、WSGIの作法でDjangoに渡し、Djangoの返事をWebサーバーへ返す、という橋渡しを担います。

流れを図にすると次の通りです（※説明用の簡易図）。

コピー

```
ブラウザ
   │ HTTPリクエスト
   ▼
[ Webサーバー / リバースプロキシ ]   （例：Azureのフロント、Nginx 等）
   │ WSGI
   ▼
[ Gunicorn ]   ← gunicorn==21.2.0（複数の作業員プロセスでさばく）
   │ WSGIの作法でアプリを呼ぶ
   ▼
[ Django アプリ ]   ← settings.py の WSGI_APPLICATION が入口
   │
   ▼
（DB・処理）→ レスポンスを逆ルートで返す
```

`settings.py` のこの1行が「WSGIで呼ばれる入口」を指定しています。

コピー

```
# Gunicornはこの application を呼び出してDjangoを起動する
WSGI_APPLICATION = 'hozen_another.wsgi.application'
```

これは「`hozen_another/wsgi.py` の中の `application` という変数を入口にせよ」という意味です。Gunicornはここを掴んでDjangoを動かします。

8.3 WSGIとASGIの違い（補足）

> **用語: ASGI（アスギー：Asynchronous Server Gateway Interface）とは？** WSGIの「非同期対応版」。WebSocket（双方向のリアルタイム通信）など、WSGIでは扱いにくい用途に対応した新しい規格。

規格	得意なこと	本アプリ
WSGI	普通のリクエスト→レスポンス型のWeb	**これを使用**（`WSGI_APPLICATION` 設定あり）
ASGI	WebSocket等のリアルタイム双方向通信	使っていない（リアルタイム機能がないため不要）

本アプリは「画面を開く → データを取る → 表示する」という素直なWebなので、**WSGI（Gunicorn）で十分** です。リアルタイムチャットのような機能を将来足すならASGIを検討します。

> **なぜ runserver ではダメなの？** `python manage.py runserver` は開発用の簡易サーバーで、同時アクセスに弱く、本番の負荷に耐えられません。本番は **Gunicorn**（複数の作業員プロセスで同時にさばける）を使うのが定石です。

9. index.tsx から見るフロント部品の使われ方

フロントの部品が **どこで効いているか** を、入口ファイル `frontend/src/index.tsx` で確認します。これは「フロントの起動スイッチ」です。

9.1 冒頭のimport（部品の取り込み）

> **▼ このコードがやること（先に日本語で）**：アプリを動かすのに必要な部品（React本体・ReactDOM・ルーター・共通CSS・各画面）を、ファイルの先頭でまとめて「取り込み（import）」しています。

コピー

```
// React本体（画面の組み立て役）
import React from 'react';
// ReactDOM（画面をブラウザに描く役）
import ReactDOM from 'react-dom/client';
// ルーター（URLと画面の対応付け）
import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom";
// 共通CSS（全画面の見た目の土台）
import './styles/global.css';
// ↓ここから下は各画面の部品（自作）
import Login from './MainPage/Login';
import Help from './MainPage/Help';
import MainMenu from './MainPage/MainMenu';
// …（中略：50近い画面を1つずつimport）…
import AdministratorHistoryDetail from './AdministratorPage/AdministratorHistoryDetail';
```

ここで `package.json` の **react / react-dom / react-router-dom** が実際に使われているのが分かります。

> **用語: import（インポート：取り込み）とは？** 別ファイルや部品の機能を「このファイルでも使えるように呼び込む」命令。料理でいえば「冷蔵庫（部品集）から必要な材料を作業台に出す」操作です。
>
> **`BrowserRouter as Router` の `as` とは？** 取り込んだ `BrowserRouter` を、このファイル内では短く `Router` という別名で呼ぶ、という意味です。

9.2 ページタイトルの自動切り替え（useLocation の活用）

> **▼ このコードがやること（先に日本語で）**：「今どのURLを開いているか」を監視し、URLごとにブラウザのタブのタイトル（例：「ログイン - 業務工数システム」）を自動で書き換えています。

コピー

```
// App という画面全体の親部品を定義
const App: React.FC = () => {
  // react-routerの機能で「現在のURL情報」を取得
  const location = useLocation();

  // 「URLが変わるたびに」実行する処理を登録
  React.useEffect(() => {
    // URL → タイトル文字列 の対応表（オブジェクト）
    const routeTitles: { [key: string]: string } = {
      // /login を開いたらこのタイトル
      "/login": "ログイン - 業務工数システム",
      "/help": "ヘルプ - 業務工数システム",
      "/": "Main Menu - 業務工数システム",
      // …（中略：全URLぶんの対応）…
      "/manager-history-detail/:id": "データ操作履歴詳細 - 業務工数システム",
    };

    // 対応表にあればそのタイトル、なければ既定値
    document.title = routeTitles[location.pathname] || "業務工数システム";
  // [location] = 「locationが変わったら再実行」の指示
  }, [location]);
```

ここで `react-router-dom` の `useLocation` と、React の `useEffect`（副作用フック）が使われています。`package.json` の **react** と **react-router-dom** の具体的な活躍場面です。

> **用語: useEffect（ユーズエフェクト）とは？** Reactの「ある条件のときに、画面描画とは別の処理（副作用）を走らせる」しくみ。ここでは「URLが変わったらタブのタイトルを変える」という副作用を担っています（Reactの章で詳説）。

9.3 ルーティング本体（Route の列）

> **▼ このコードがやること（先に日本語で）**：「このURLなら、この画面部品を表示する」という対応を、`<Route>` を50個近く並べて定義しています。これが react-router-dom の本領です。

コピー

```
  return (
    {/* ルート（対応）をまとめる箱 */}
    <Routes>
      {/* /login → Login画面 */}
      <Route path="/login" element={<Login />} />
      {/* /help → Help画面 */}
      <Route path="/help" element={<Help />} />
      {/* / → メインメニュー */}
      <Route path="/" element={<MainMenu />} />
      {/* 以下、各機能の画面が続く */}
      <Route path="/kosu-menu" element={<KosuMenu />} />
      {/* …（中略：工数・定義・人員・班員・問い合わせ・管理者の各画面）… */}
      <Route path="/manager-history-detail/:id" element={<AdministratorHistoryDetail />} />
    </Routes>
  );
};
```

- `path="..."` … URLのパターン。`:id` は「ここに数字やID等が入る可変部分」という印（パスパラメータ）。
- `element={<○○ />}` … そのURLで表示する画面部品。

9.4 アプリの起動（createRoot と render）

> **▼ このコードがやること（先に日本語で）**：HTMLの中の `id="root"` の場所を見つけ、そこにReactアプリ全体を描き込みます。これが「フロントの最終的な起動スイッチ」です。

コピー

```
// HTMLの<div id="root">を画面の土台にする
const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement);

// その土台に描く
root.render(
  {/* 開発中に潜在バグを警告してくれる「厳格モード」で包む */}
  <React.StrictMode>
    {/* ルーター（URL監視）で包む */}
    <Router>
      {/* アプリ本体を表示 */}
      <App />
    </Router>
  </React.StrictMode>
);
```

- `ReactDOM.createRoot(...)` … `package.json` の **react-dom** の機能。
- `<Router>` … **react-router-dom** の `BrowserRouter`（別名Router）。URL監視を有効化。

- `<React.StrictMode>` … **react** の機能。開発時のみ余分なチェックを走らせ、危ない書き方を警告する「安全モード」。

> **用語: id="root" とは？** `frontend/build/index.html` の中にある空の `<div id="root"></div>`。Reactはこの「空の額縁」の中に、アプリ全体を描き込みます。HTMLは入口の1枚だけで、中身はすべてReactが作るのが「SPA（シングルページアプリ）」の特徴です（Reactの章で詳説）。
>
> ⚠️ **注目: ここに `web-vitals`（reportWebVitals）の呼び出しがありません。** CRAの標準テンプレートなら `reportWebVitals()` を呼ぶ行があるのですが、本アプリの `index.tsx` には **その行が存在しません。** つまり `reportWebVitals.ts` も `web-vitals` も実質的に使われていない、という§10の結論の決定的な証拠です。

---

10. 正直な棚卸し：使われていない疑いのある部品

ここが、この章で **後任者にとって最も価値の高い** セクションです。本アプリには「部品リストに書いてあるのに、実際のコードでは使われていない疑いのある部品」が多数あります。各部品について、実際にコード全体を検索した結果を根拠とともに表にまとめます。

> ⚠️ **重要な前置き:** 「未使用の疑い」は、**ソースコードを全文検索して直接importが見つからなかった** という客観的事実に基づきます。ただし、(a) 設定ファイル経由で間接的に効いている、(b) 別の部品が内部で依存している、(c) 検索に引っかからない特殊な使い方をしている、という可能性はゼロではありません。**いきなり消さず、§12の安全手順で確認してから** 整理してください。

10.1 バックエンド（requirements.txt）の棚卸し

部品	バージョン	調査結果（根拠）	判定
`xlwings`	0.30.13	`import xlwings` がコードに見つからない。Excel処理は openpyxl が担当。さらにxlwingsはExcelアプリ常駐が前提でLinuxサーバーで動かない	✕ 未使用の疑い
`beautifulsoup4`	4.12.2	`import bs4` / `from bs4` が見つからない。スクレイピング機能なし	✕ 未使用の疑い
`python-Levenshtein`	0.26.1	`import Levenshtein` が見つからない。類似度計算の処理なし	✕ 未使用の疑い
`python-json-logger`	2.0.7	`settings.py` のLOGGINGは `simple`（`{message}`）でJSONフォーマッタ未使用。直接importもなし	✕ 未使用の疑い
`django-crispy-forms`	2.1	INSTALLED_APPS未登録。テンプレート時代のフォーム整形用で、React化により不要	✕ 未使用の疑い
`django-filter`	23.5	INSTALLED_APPS未登録、DRFのフィルタ設定もなし	✕ 未使用の疑い
`django-widget-tweaks`	1.5.0	INSTALLED_APPS未登録、テンプレート未使用	✕ 未使用の疑い
`django-widgets-improved`	1.5.0	INSTALLED_APPS未登録、テンプレート未使用	✕ 未使用の疑い
`django-bootstrap4`	23.3	INSTALLED_APPSに登録は残るが、React画面では未使用（旧テンプレート用）	△ 設定のみ残存
`django-bootstrap-datepicker-plus`	5.0.5	INSTALLED_APPSに登録は残るが、React画面では未使用	△ 設定のみ残存
`python-dateutil`	2.8.2	直接importなし。pandas等の間接依存の可能性	△ 間接依存の可能性
`python-dotenv`	1.0.0	settings.pyは django-environ を使用。直接importなし（環境変数読み込みの重複）	△ 重複の疑い

10.2 フロントエンド（package.json）の棚卸し

部品	バージョン	調査結果（根拠）	判定
`@fullcalendar/daygrid`	6.1.19	`@fullcalendar/...` の import が src に1件もない	✕ 未使用の疑い
`@fullcalendar/interaction`	6.1.19	同上	✕ 未使用の疑い
`@fullcalendar/react`	6.1.19	同上	✕ 未使用の疑い
`@fullcalendar/timegrid`	6.1.19	同上	✕ 未使用の疑い
`react-calendar`	6.0.0	`from 'react-calendar'` の import が見つからない。カレンダーは自前実装（§11）	✕ 未使用の疑い
`dayjs`	1.11.13	`from 'dayjs'` の import が見つからない。日付処理は date-fns を採用	✕ 未使用の疑い
`web-vitals`	2.1.4	`reportWebVitals.ts` 内でのみ参照。だが reportWebVitals 自体が index.tsx から呼ばれていない（§9.4）	✕ 未使用の疑い

10.3 棚卸しのまとめ（一目で）

> **▼ ここまでで分かること（先に日本語で）**：「実際に使われている中核部品」と「未使用の疑いがある部品」をきれいに線引きできました。後任者は、まず中核部品を理解すれば十分で、未使用候補は「触らないか、後で安全に整理する」対象です。

- **バックエンドの中核（必ず理解すべき）**：Django / DRF / Django-Q / cors-headers / environ / jsonfield / psycopg2 / gunicorn / whitenoise / openpyxl / pandas
- **フロントの中核（必ず理解すべき）**：react / react-dom / react-router-dom / axios / typescript / react-scripts / MUI(+emotion) / x-date-pickers(+date-fns) / chart.js系 / （開発）vitest・RTL・jsdom
- **未使用の疑い（整理候補）**：xlwings / beautifulsoup4 / python-Levenshtein / python-json-logger / crispy-forms / django-filter / widget-tweaks / widgets-improved / FullCalendar4点 / react-calendar / dayjs / web-vitals、設定だけ残るbootstrap4・datepicker-plus

なぜこんなに残っているの？ このアプリは「Djangoテンプレートで作っていた旧版」→「Reactに作り替えた現行版」へと進化した歴史があります。テンプレート時代の部品（Bootstrap・crispy・widget系・datepicker）や、移行時にいろいろ試した跡（FullCalendar・react-calendar・dayjs）が **掃除されずに残っている** のです。これは珍しいことではなく、多くの実プロジェクトで起きる「技術的負債（ツケ）」です。

11. カレンダーは自前実装という事実

§10で「FullCalendarもreact-calendarも使われていない」と書きました。では、本アプリのカレンダー画面（勤務入力など）は何で作られているのでしょうか。答えは **自作の `generateCalendar` 関数** です。

調査の結果、`generateCalendar` は次のファイルに存在します。

コピー

```
frontend/src/KosuPage/KosuCalendar.tsx
```

▼ **ここで分かること（先に日本語で）**：本アプリのカレンダーは、外部のカレンダー部品（FullCalendar等）に頼らず、月の日付の並びを **自前のロジックで計算して** 作っています。だから package.json のカレンダー部品はどれも実際には呼ばれていないのです。

用語: 自前実装（じまえじっそう）とは？ 既製の部品を使わず、自分でコードを書いて機能を作ること。`generateCalendar` は「その月が何日まであるか」「1日が何曜日から始まるか」を計算し、日付のマス目（グリッド）を組み立てる自作関数です。

なぜ自前にしたのか（推測）：FullCalendar等は高機能ですが、本アプリのカレンダーは「日付を選んで勤務情報を入れる」というシンプルな用途。それなら自前で作った方が **軽く・思い通りに** できると判断したと考えられます。試しに部品を入れてみた（package.jsonに残った）が、最終的に自前実装を選んだ、という流れが自然です。

⚠️ **後任者へ**: カレンダーの見た目・挙動を変えたいときは、FullCalendarの設定を探しても無駄です。**`KosuPage/KosuCalendar.tsx` の `generateCalendar` 関数を直接読んで** ください。同様にチーム側のカレンダーは `TeamPage/TeamCalendar.tsx` を確認します（フロントの画面解説の章で詳述）。

12. 不要な依存を安全に整理する手順

「未使用の疑いがある部品を消したい」となったとき、**いきなり消すのは危険** です。間接依存や設定経由の利用を壊す恐れがあるからです。ここでは安全な手順を示します。

> ⚠️ **大原則: 整理は「専用ブランチ」で行い、消すたびに必ず動作確認する。** 1つ消して動作確認、また1つ消して動作確認、と **小刻みに** 進めます。一度に大量に消してはいけません。

12.1 事前準備（バックアップと作業ブランチ）

> ▼ **このコマンドがやること（先に日本語で）**：整理専用の作業用ブランチ（枝）を作って、そこで作業します。失敗しても本流（master）に影響が出ないようにする安全策です。

コピー

```
# 整理用の新しいブランチを作って移動する
git switch -c chore/cleanup-unused-deps
```

> **用語: ブランチ（branch：枝）とは？** 本流のコードを傷つけずに、別の枝で実験するためのGitの機能。失敗したら枝ごと捨てれば本流は無傷です（Gitの章で詳説）。

12.2 バックエンド：1つずつ消して確認

手順（※説明用の安全フロー）：

1. `requirements.txt` から **1行だけ** 削除する（例：`xlwings==0.30.13` を消す）。
2. 仮想環境を作り直すか、その部品をアンインストールする。`bash # その部品だけ取り除く pip uninstall xlwings`
3. アプリを起動して動作確認する。`bash # 設定の整合性チェック（まず軽く確認） python manage.py check # 実際に起動して画面を触る python manage.py runserver # 非同期タスク（バックアップ等）も起動して試す python manage.py qcluster`
4. テストを流す（バックエンドにテストがあれば）。`bash # kosuアプリのテストを実行 python manage.py test kosu`
5. 問題なければコミット。問題が出たら **その1行を元に戻す**。

> ▼ **こう表示されれば成功:** `python manage.py check` で `System check identified no issues (0 silenced).` と出れば、設定上の問題はありません。
>
> ⚠️ **INSTALLED_APPSに登録が残っている部品（bootstrap4・bootstrap_datepicker_plus）を消すときは、requirements.txt だけでなく `settings.py` の登録行も同時に消す** 必要があります。片方だけ消すと「登録したのに部品がない（または逆）」でエラーになります。`builtins` の `'bootstrap4.templatetags.bootstrap4'` や `BOOTSTRAP4 = {...}` も合わせて確認してください。

12.3 フロントエンド：1つずつ消して確認

> **▼ このコマンドがやること（先に日本語で）**：フロントの未使用部品を1つだけ取り除き、ビルドとテストが通るか確認します。

1. 1つだけアンインストール（`frontend/` で実行）。`bash # 1部品だけ取り除く（package.jsonからも自動で消える） npm uninstall @fullcalendar/daygrid`
2. ビルドが通るか確認。`bash # 本番ビルドが成功すればOK npm run build`
3. テストを流す。`bash # テストが全部通ればOK npm run test:run`
4. 開発サーバーで画面を実際に触る。`bash # localhost:3000 で各画面（特にカレンダー・グラフ）を確認 npm start`
5. 問題なければコミット。出たら元に戻す（`npm install <部品名>@<元のバージョン>`）。

> **▼ こう表示されれば成功:** `npm run build` の最後に `Compiled successfully.` 、`npm run test:run` で全テストが緑（pass）になれば、その部品は安全に消せたと判断できます。

12.4 未使用部品を見つける補助ツール（参考）

機械的に「使われていない依存」を洗い出す補助ツールもあります（※参考情報。導入は任意）。

- フロント：`depcheck`（未使用のnpm依存を検出）、`npx depcheck` で実行。
- バック：`pip-check` や `deptry` 等。

> ⚠️ これらのツールも **完璧ではありません**（設定経由・間接依存を誤検出することがある）。あくまで「あたりを付ける道具」として使い、最終判断は §12.2 / §12.3 の「1つずつ消して動作確認」で行ってください。

12.5 整理の優先順位（おすすめ）

優先	対象	理由
高（消しやすい）	FullCalendar 4点 / react-calendar / dayjs / web-vitals	フロントで import 0件。ビルド・テストで即確認できる
高	xlwings / beautifulsoup4 / python-Levenshtein	import 0件で、機能との結びつきもない
中	django-filter / crispy-forms / widget-tweaks / widgets-improved	INSTALLED_APPS未登録なので影響は小さいが、念のため検索を再確認
中	python-json-logger	LOGGING設定がsimpleであることを再確認してから
低（慎重に）	bootstrap4 / bootstrap_datepicker_plus	settings.py の登録も同時に外す必要あり。手順が増える
触らない	python-dateutil / setuptools / python-dotenv	間接依存・基盤の可能性。消すと別部品が壊れる恐れ。最後に慎重判断

> **なぜ「触らない」列があるの？** `python-dateutil` は pandas が、`setuptools` は多くの部品が、内部で必要とします。消すと一見無関係な部品が動かなくなる「巻き添え事故」が起きます。確証がない限り **残しておくのが安全** です。

13. トラブルシューティング

症状	原因	対処
`pip install -r requirements.txt` でエラー	Pythonのバージョン不一致、ネット遮断、`psycopg2-binary` のビルド失敗	Pythonのバージョンを確認。社内プロキシならpipのプロキシ設定を行う
`npm install` が途中で止まる	Node.jsのバージョン不一致、ネット遮断、`package-lock.json` との不整合	Node.jsのバージョンを確認。`package-lock.json` を消さない。社内なら `npm config set proxy`
`ModuleNotFoundError: No module named 'xxx'`	その部品が未インストール、または整理で消しすぎた	`pip install xxx`（または `requirements.txt` を元に戻す）
`Module not found: Can't resolve 'xxx'`（フロント）	npm部品が未インストール、または消しすぎた	`npm install xxx`（または `package.json` を元に戻す）
部品を消したらカレンダーが消えた／壊れた	実は使っていた部品を消した	`git restore` で戻す。カレンダーは自前 `generateCalendar`（§11）なので、壊れたのは別原因かも
`npm run build` は通るのに本番で表示が崩れる	静的ファイルの収集漏れ（WhiteNoise関連）	`python manage.py collectstatic` を実行し直す（デプロイの章参照）
バージョンを上げたら急に動かない	メジャーバージョンが上がって破壊的変更が入った（§1.1）	そのバージョンを元に戻す。メジャーアップは慎重に
`eject` してしまった	CRAの隠し設定が全部展開され、元に戻せない	`git` で `eject` のコミットを取り消す（事前にコミットしておくこと）
「これ使ってる？」と毎回不安になる	棚卸しが頭に入っていない	§10の表をブックマーク。`Grep` でその部品名をsrc 全体検索して確かめる習慣を

14. 演習問題

> 答えはこの章の本文中にあります。手を動かして確認してください。

問1. バージョン `4.2.7` の「4」「2」「7」はそれぞれ何と呼ばれ、どんなときに数字が増えますか。

問2. `requirements.txt` の `==` と、`package.json` の `^` は、バージョン指定の意味がどう違いますか。

問3. 本アプリのデータベース（PostgreSQL）にDjangoが接続するために必須の部品は何ですか。役割を「通訳」というたとえで説明してください。

問4. `django-q`（Django-Q）は何のための部品ですか。「画面を固まらせない」という観点で、なぜ必要か説明してください。

問5. `date-fns` と `dayjs` は同じ目的の部品です。本アプリで **実際に使われている** のはどちらですか。そう判断できる根拠（どのファイルのどんな import）を挙げてください。

問6. FullCalendarやreact-calendarが `package.json` にあるのに、本アプリのカレンダー画面はそれらを使っていません。では何で作られていますか。関数名とファイル名を答えてください。

問7. `index.tsx` の最後で `web-vitals`（reportWebVitals）が **使われていない** と判断できる決定的な理由は何ですか。

問8. `bootstrap4` を整理（削除）するとき、`requirements.txt` 以外に手を入れる必要があるファイルと箇所を、`settings.py` の中から3つ挙げてください。

問9. WSGIとは何の規格ですか。Gunicornはその中でどんな役割を果たしますか。「コンセントの形」のたとえを使って説明してください。

問10. 未使用と疑われる部品を整理するとき、「いきなり全部消してはいけない」のはなぜですか。`python-dateutil` を例に説明してください。

問11（実践）. 自分のパソコンで `frontend/` に移動し、`Grep`（または `npx depcheck`）を使って `dayjs` が本当に import されていないか自分で確かめてください。確認できたら、§12.3 の手順で安全に削除を試し、`npm run build` と `npm run test:run` が通ることを確認してください（うまくいかなければ元に戻すこと）。

---

15. この章のまとめ

- **ライブラリ（部品）** は、誰かが作った再利用可能なプログラム。自分で全部書かずに済むよう「借りて」使う。倉庫はPyPI（Python）とnpm（JavaScript）。
- 部品リストは `requirements.txt`（pip）と `package.json`（npm）。`pip install -r requirements.txt` と `npm install` で一括復元できる。
- **バージョン番号** は `メジャー.マイナー.パッチ`。`==` は完全固定、`^` は「メジャーは上げない範囲で新しくてOK」。`package-lock.json` も必ず引き継ぐ。
- **バックエンドの中核**：Django / DRF / Django-Q / cors-headers / django-environ / jsonfield / psycopg2 / Gunicorn / WhiteNoise。データ処理に openpyxl / pandas（実コードで使用確認）。
- **フロントの中核**：React / ReactDOM / React Router DOM / axios / TypeScript / react-scripts(CRA) / MUI(+emotion) / x-date-pickers(+date-fns) / chart.js系。テストは Vitest / RTL / jsdom。
- **WSGI** は「PythonアプリとWebサーバーの会話の規格」。**Gunicorn** が本番でその橋渡しを担う。`settings.py` の `WSGI_APPLICATION` が入口。本アプリはWSGIで十分（ASGIは未使用）。

- **正直な棚卸し**：xlwings / beautifulsoup4 / python-Levenshtein / python-json-logger / crispy-forms / django-filter / widget系 / FullCalendar / react-calendar / dayjs / web-vitals は **未使用の疑い**。bootstrap4・datepicker-plusは **設定にだけ残存**。
- **カレンダーは自前実装**（`KosuPage/KosuCalendar.tsx` の `generateCalendar`）。外部カレンダー部品は実際には使われていない。
- **整理は専用ブランチで、1つずつ消して動作確認**。`python-dateutil`・`setuptools`・`python-dotenv` は間接依存・基盤の可能性があり、確証なしに消さない。

> 次は **第12章: テストの書き方と実行** に進みます。本章で登場した Vitest / React Testing Library / jsdom が、実際にどんなテストコードを書き、どう実行されるのかを、本アプリのテストファイルを1行ずつ読みながら学びます。